Helm Chart Orchestration Guide

1. Architecture Overview

A Helm chart is the structural blueprint of your application. It encapsulates all Kubernetes resources into a versioned, reusable package.

Project Root

- **`Chart.yaml`** : The identity of your application. Contains name, version, and metadata.
- **`values.yaml`** : The control panel. This is where you toggle features, set resource limits, and define environment-specific variables without touching the logic.
- **`.helmignore`** : Keeps your chart clean by excluding unnecessary build or source files.

The `templates/` Directory

Templates use the **Go Templating Engine** to inject data from `values.yaml` into Kubernetes manifests.

- **`deployment.yaml`** : Defines how your application runs (replicas, images, environment variables).
- **`service.yaml`** : Exposes your application internally or externally via a stable IP/DNS.
- **`ingress.yaml`** : Manages external access (HTTP/HTTPS) to your services, often with TLS and routing rules.
- **`_helpers.tpl`** : A utility library for reusable naming conventions and labels (prevents DRY — Don't Repeat Yourself).

---

2. High-Level Integration Patterns

When building for different environments (Production, Staging, Dev), you should avoid hardcoding. Use the `{{ .Values.x }}` pattern to keep your templates dynamic.

Reusable Networking

Instead of hardcoding hosts in your Ingress, use:

```
host: {{ .Values.ingress.host }}
```

Resource Management

Always define `resources` in your `values.yaml` to prevent a single pod from consuming all node capacity:

```
resources:
  limits:
    cpu: 1000m
    memory: 1Gi
  requests:
    cpu: 500m
    memory: 512Mi
```

---

3. Case Study: GPU-Accelerated Deployments

For AI, Computer Vision, or RAG workloads, your deployment needs specific scheduling constraints (Tolerations & Affinity) to access NVIDIA hardware.

Values Configuration (`values.yaml`)

Configure your GPU nodes (e.g., via Karpenter) by ensuring your pods only land on specialized high-compute hardware.

```yaml
# Default CPU scheduling
tolerations:
  - key: "karpenter-cpu"
    operator: "Equal"
    effect: "NoSchedule"
    value: "true"
  - key: "project"
    operator: "Equal"
    effect: "NoSchedule"
    value: "cosmichorizon"

# Specialized GPU scheduling
tolerationsGPU:
  - key: "nvidia.com/gpu"
    operator: "Equal"
    effect: "NoSchedule"
    value: "true"
  - key: "project"
    operator: "Equal"
    effect: "NoSchedule"
    value: "cosmichorizon"

# Keep vision/RAG components on the same node for speed
affinityGPU:
  podAffinity:
    requiredDuringSchedulingIgnoredDuringExecution:
    - labelSelector:
        matchExpressions:
        - key: component
          operator: In
          values:
          - rag-core
          - parser
          - vision
      topologyKey: "kubernetes.io/hostname"

# Vision Service (GPU Enabled)
vision:
  replicaCount: 1
  image:
    repository: harbor.i2cv.io/cosmichorizon/airbud-vision
    tag: latest
  gpu:
    enabled: true
    count: 1
  resources:
    limits:
      nvidia.com/gpu: 1
      cpu: "2"
      memory: "4Gi"
    requests:
      cpu: "1"
      memory: "2Gi"
```

Template Implementation (`deployment-vision.yaml`)

The deployment manifest must request the GPU device and apply the corresponding scheduling logic.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ include "airbud.fullname" . }}-vision
  labels:
    {{- include "airbud.labels" . | nindent 4 }}
    component: vision
spec:
  replicas: {{ .Values.vision.replicaCount }}
  selector:
    matchLabels:
      {{- include "airbud.selectorLabels" . | nindent 6 }}
      component: vision
  template:
    metadata:
      labels:
        {{- include "airbud.selectorLabels" . | nindent 8 }}
        component: vision
    spec:
      containers:
        - name: vision
          image: "{{ .Values.vision.image.repository }}:{{ .Values.vision.image.tag }}"
          resources:
            {{- toYaml .Values.vision.resources | nindent 12 }}
          # ... (ports, env, probes)

      # GPU-Specific Scheduling
      {{- with .Values.affinityGPU }}
      affinity:
        {{- toYaml . | nindent 8 }}
      {{- end }}
      {{- with .Values.tolerationsGPU }}
      tolerations:
        {{- toYaml . | nindent 8 }}
      {{- end }}
```

---

4. Quick Start: Complete Sample Project

The following files represent a fully functional Helm chart named `gpu-app`. This configuration is designed to run a GPU-accelerated vision service with secure image pulling.

Chart.yaml

The metadata for your chart.

```
apiVersion: v2
name: gpu-app
description: A production-ready GPU-accelerated application
type: application
version: 1.0.0
appVersion: "1.0.0"
```

- **`apiVersion`**: `v2` is required for Helm 3 projects.
- **`name`**: The identity of your chart (must match the directory name).
- **`version`**: The version of the *chart package*. Increment this when you change the Helm logic.

- `appVersion`: The version of the *underlying application* (e.g., your Docker tag).

values.yaml

The configuration layer. This includes `imagePullSecrets` for private registries and specialized GPU scheduling.

```yaml
# Global identity
imagePullSecrets:
  - name: internal-harbor-registry

# Resource scheduling
tolerations:
  - key: "project"
    operator: "Equal"
    effect: "NoSchedule"
    value: "cosmichorizon"

tolerationsGPU:
  - key: "nvidia.com/gpu"
    operator: "Equal"
    effect: "NoSchedule"
    value: "true"

# Vision Service Configuration
vision:
  replicaCount: 1
  image:
    repository: harbor.i2cv.io/cosmichorizon/vision-model
    tag: latest
  resources:
    limits:
      nvidia.com/gpu: 1
      cpu: "2"
      memory: "4Gi"
    requests:
      cpu: "1"
      memory: "2Gi"

service:
  type: ClusterIP
  port: 8080
```

- `imagePullSecrets`: Grants Kubernetes the credentials needed to pull images from private registries.
- `tolerations`: Allows pods to be scheduled on "tainted" nodes (e.g., specialized GPU instances).
- `resources`: Controls CPU/RAM allocation. **Limits** are hard caps, while **Requests** are guaranteed minimums.

templates/_helpers.tpl

Standardized naming conventions.

```templates/_helpers.tpl
{{- define "gpu-app.name" -}}
{{- default .Chart.Name .Values.nameOverride | trunc 63 | trimSuffix "-" }}
{{- end }}

{{- define "gpu-app.fullname" -}}
{{- if .Values.fullnameOverride }}
{{- .Values.fullnameOverride | trunc 63 | trimSuffix "-" }}
{{- else }}
{{- $name := default .Chart.Name .Values.nameOverride }}
{{- printf "%s-%s" .Release.Name $name | trunc 63 | trimSuffix "-" }}
{{- end }}
{{- end }}
```

```
{{- define "gpu-app.labels" -}}
helm.sh/chart: {{ .Chart.Name }}-{{ .Chart.Version | replace "+" "_" }}
{{ include "gpu-app.selectorLabels" . }}
app.kubernetes.io/version: {{ .Chart.AppVersion | quote }}
app.kubernetes.io/managed-by: {{ .Release.Service }}
{{- end }}

{{- define "gpu-app.selectorLabels" -}}
app.kubernetes.io/name: {{ include "gpu-app.name" . }}
app.kubernetes.io/instance: {{ .Release.Name }}
{{- end }}
```

- `define` : Creates a named, reusable snippet (template).
- `include` : Calls a snippet and injects its output.
- `trunc 63` : Ensures names stay within the 63-character Kubernetes DNS limit.
- `selectorLabels` : Critical for connecting Services and Ingresses to the correct Pods.

templates/deployment.yaml

The core orchestration manifest.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ include "gpu-app.fullname" . }}
  labels:
    {{- include "gpu-app.labels" . | nindent 4 }}
spec:
  replicas: {{ .Values.vision.replicaCount }}
  selector:
    matchLabels:
      {{- include "gpu-app.selectorLabels" . | nindent 6 }}
  template:
    metadata:
      labels:
        {{- include "gpu-app.selectorLabels" . | nindent 8 }}
        component: vision
    spec:
      imagePullSecrets:
        {{- toYaml .Values.imagePullSecrets | nindent 8 }}
      containers:
        - name: vision
          image: "{{ .Values.vision.image.repository }}:{{ .Values.vision.image.tag }}"
          ports:
            - containerPort: 8080
          resources:
            {{- toYaml .Values.vision.resources | nindent 12 }}

      # Node Scheduling
      {{- with .Values.tolerationsGPU }}
      tolerations:
        {{- toYaml . | nindent 8 }}
      {{- end }}
```

- `matchLabels` : The "glue" that allows the Deployment to track and manage its Pods.
- `toYaml` / `nindent` : These work together to inject values from `values.yaml` while respecting exact YAML indentation requirements.
- `tolerations` **(Bottom Section)**: Specifically injected here to ensure the Pod is authorized to land on GPU-enabled nodes.

templates/service.yaml

Internal networking.

```
apiVersion: v1
kind: Service
metadata:
  name: {{ include "gpu-app.fullname" . }}
  labels:
    {{- include "gpu-app.labels" . | nindent 4 }}
spec:
  type: {{ .Values.service.type }}
  ports:
    - port: {{ .Values.service.port }}
      targetPort: 8080
      protocol: TCP
  selector:
    {{- include "gpu-app.selectorLabels" . | nindent 4 }}
```

- **`port`** : The port number exposed on the *Service IP*.
- **`targetPort`** : The port number the *Application* is actually listening on inside the container.
- **`selector`** : Routes traffic to any Pod that matches these labels (typically defined in `_helpers.tpl`). ```